LR2-REDUX · FIELD NOTES

Breadboard Wiring Worksheet

Bench-prototype wiring for the ESP32-WROOM + L298N control board — the two new hall-effect sensors get bench-tested here with a hand-held magnet before permanent installation; the stock motor and weight switch connect through the original harness, using the pin identities confirmed in the harness pinout worksheet.

Rev. 2026-07-05 (3-LED status: green/yellow/red)

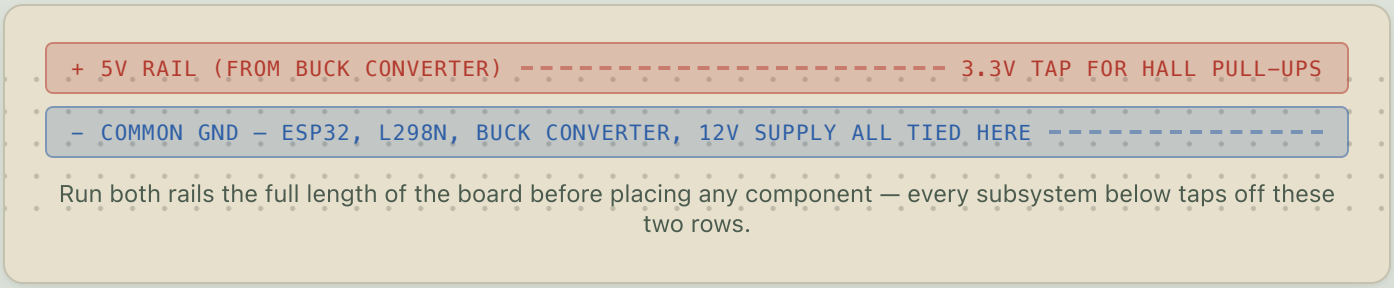
Companion to: harness pinout worksheet

Source: README.md pinout + BOM

10
ESP32 GPIOs assigned

2
rails to build: 5V logic, 12V motor

3 / 3
bring-up checks complete (2026-07-05)



Run both rails the full length of the board before placing any component — every subsystem below taps off these two rows.

Power tree

Build and verify this before wiring anything else

Stock 12V wall adapter → 2A inline fuse → 12V rail → L298N 12V in, buck converter in

Buck converter (12V→5V, 1A+) → 5V rail → ESP32 VIN, both hall sensor VCC pins, L298N logic-5V

GND — ESP32 GND, L298N GND, buck converter GND, and 12V supply GND all tied to one common bus

Do not power the ESP32 from the L298N's onboard 5V regulator tapped directly off the 12V rail — motor switching noise rides straight into it. Use the dedicated buck converter for ESP32 + hall sensor VCC, kept

electrically separate from the L298N's own logic-supply pin even though both read "5V."

ESP32 pin assignment

All functions below share the common GND bus

26	**Motor IN1**	L298N direction input
27	**Motor IN2**	L298N direction input
25	**Motor ENA**	PWM speed — ledc channel 0, 5kHz
34	**Hall — Home**	input-only pin, needs the external 10kΩ pull-up (no internal pull available)
35	**Hall — Dump**	input-only pin, needs the external 10kΩ pull-up (no internal pull available)
32	**Weight / cat switch**	active-low, internal INPUT_PULLUP — via stock harness pin 6 (physically split from pin 7, see below)
14	**Anti-pinch switch**	**active-HIGH** (normally-closed, opposite of every other input here), internal INPUT_PULLUP — via stock harness pin 7
33	**Manual cycle button (optional)**	active-low, internal INPUT_PULLUP
4	**Status LED — green**	discrete LED, see Status LED panel below
16	**Status LED — yellow**	discrete LED, see Status LED panel below
17	**Status LED — red**	discrete LED, see Status LED panel below

Subsystem wiring

One panel per breadboard section

Hall sensors (×2)

BENCH-MOUNTED

5V rail → Hall VCC

Hall OUT → GPIO 34 (Home) / 35 (Dump)

Motor driver (L298N)

STOCK HARNESS

GPIO 26 → IN1, GPIO 27 → IN2

GPIO 25 → ENA (PWM)

```
Hall OUT → 10kΩ → 3.3V (pull-up)
Hall GND → GND
```

Pull-up goes to **3.3V, not 5V** — ESP32 GPIOs aren't 5V-tolerant. The stock sensors turned out to be **Allegro A1101EUA-T** (confirmed by opening the sensor package), datasheet VCC range 3.8-24V, open-drain output — 5V is correct and confirmed for VCC, pull the output up to 3.3V only.

Both sensors mount directly on the breadboard with a hand-held test magnet for bring-up — not wired through the stock harness. The stock harness's hall-sensor pins (7-pin connector, pins 2–5) are for reference only; see the harness worksheet.

```
12V rail → L298N 12V in
5V rail → L298N logic-5V, GND → L298N GND
L298N OUT1/OUT2 → stock motor leads (harness worksheet, 4-pin connector, pins 3 & 4)
```

Only one motor direction is used in firmware — matches the stock LR2, which always rotates the globe the same way through a full cycle. Confirm your breakout board includes the four flyback diodes most L298N modules ship with.

Weight, anti-pinch + button

STOCK HARNESS · SPLIT DONE 2026-07-05

```
Weight switch → GPIO 32 (INPUT_PULLUP)
via 7-pin harness connector, pin 6
Anti-pinch switch → GPIO 14 (INPUT_PULLUP, active-HIGH)
via 7-pin harness connector, pin 7
Manual button (optional) → GPIO 33 (INPUT_PULLUP)
```

All three use the ESP32's **internal** pull-ups — no external resistor needed, unlike the hall sensors. This was a real bug caught mid-project: an earlier firmware revision left these floating on plain `INPUT`.

Status LED (3 discrete LEDs)

GREEN + YELLOW + RED, 2026-07-05

```
GPIO 4 → ~330Ω → green LED anode → GND
GPIO 16 → ~330Ω → yellow LED anode → GND
GPIO 17 → ~330Ω → red LED anode → GND
```

Replicates the original stock board's status language exactly (from its manual): **solid green** = standby/waiting, **solid yellow** = cycling, **solid red** = cat sensor active or a weight-triggered interruption, **flashing red** = cat sensor active too long or a fault, **flashing yellow** = anti-pinch-triggered interruption specifically. See `README.md` "Status LED" for the full table.

Pins 6/7 were originally wired in series on one loop by the stock harness — confirmed by bench test, and independently corroborated by another Litter-Robot rebuild hitting the identical problem. Read as one signal, that circuit couldn't tell "cat present" from "pinch detected," and a mid-cycle pinch was completely invisible since the weight switch is already open whenever a cycle is running (no software fix exists for this — proven, not assumed). **Fixed 2026-07-05**: case opened, the internal jumper connecting the two switches cut, each given its own ground return — both now bench-verified reading fully independently. Firmware: `PIN_ANTI_PINCH`, independent `SAFETY_STOP` interlock.

Polarity bug caught during verification: the anti-pinch switch is **normally-closed** — it reads "on" at rest and "off" only when actually tripped, the opposite of every other switch here. The firmware originally assumed uniform active-low polarity, which would have made the safety interlock trigger constantly during normal use while missing a real pinch entirely. Fixed with a per-input `activeHigh` flag on `DebouncedInput` — see `CLAUDE.md` for the full story.

Three separate physical LEDs, each with its own resistor and independent GPIO — **not** a bi-color/2-channel part (an earlier pass at this briefly assumed a bi-color red+green LED faking yellow by lighting both; the user clarified the real hardware is three discrete LEDs). GPIO4/16 were already wired for the earlier two-LED design; GPIO17 is the one new pin this adds.

Breadboard build worksheet

All landed except the new red LED wire (GPIO17)

CONNECTION	ESP32 / MODULE PIN	BREADBOARD ROW	LANDED
5V rail feed	buck converter OUT+	row #	☑
3.3V pull-up tap	ESP32 3V3 pin	row #	☑
GND bus	all module GNDs	row #	☑
Hall — Home	GPIO34	row #	☑

CONNECTION	ESP32 / MODULE PIN	BREADBOARD ROW	LANDED
Hall — Dump	GPIO35	row #	☑
Motor IN1 / IN2 / ENA	GPIO26 / 27 / 25	row #	☑
Weight switch	GPIO32	row #	☑
Anti-pinch switch	GPIO14	row #	☑
Manual button	GPIO33	row #	☑
Status LED — green	GPIO4	row #	☑
Status LED — yellow	GPIO16	row #	☑
Status LED — red	GPIO17	row #	☐

Power-on sequence

Complete as of 2026-07-05

1. ☑ **Power the rails alone** — 12V in, buck converter, 5V and GND rails — with no modules plugged in yet. Meter the 5V rail before connecting anything to it.
   Why: a wiring mistake here can only damage the buck converter, not the ESP32 or sensors.
2. ☑ **Plug in the ESP32 alone**, confirm it boots (USB serial or the status LED) before adding the L298N or sensors.
   Why: isolates ESP32 bring-up from motor-driver wiring mistakes.
3. ☑ **Add the hall sensors, test with a hand-held magnet** before wiring the L298N or connecting the stock motor.
   Why: matches the project's bench-test-before-install approach — confirms sensor + pull-up wiring independent of motor power switching noise.

Source: README.md bill of materials, pinout table, and wiring notes. Stock-harness pin references (motor leads, weight/anti-pinch switches) point to the harness pinout worksheet — continuity trace 2026-07-03, hall sensor teardown + weight/anti-pinch bench testing 2026-07-04, hardware split + polarity bug fix 2026-07-05. Full narrative in this project's CLAUDE.md, "Key decisions" and "Original harness pinout" sections.